

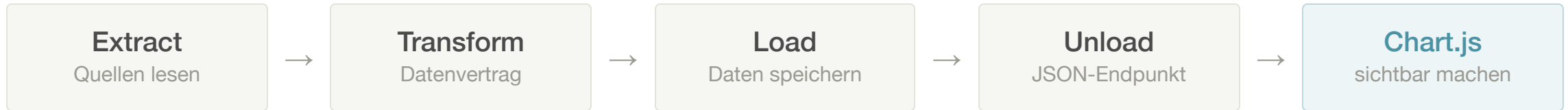
Datengrafiken

Vom JSON-Endpunkt zum Diagramm

Inhalt

- Welches Diagramm zu welcher Aussage passt
- Was Chart.js kann – und was nicht
- Von Datensätzen zu `labels` und `datasets`
- Die Daten mit `fetch()` holen
- Auf Bedienung reagieren
- Karten mit Leaflet, wenn die Frage ein Wo ist

Wo wir stehen



Die Daten sind gelesen, geputzt, gespeichert und werden ausgeliefert.

Jetzt sieht sie zum ersten Mal jemand, der euren Code nicht kennt.

Alles bisher war Vorbereitung für diesen Schritt.

Erst die Aussage, dann der Diagrammtyp

Ein Diagrammtyp ist eine Behauptung über die Daten.

Eine Linie behauptet, dass die Punkte zusammenhängen und aufeinander folgen.

Ein Balken behauptet, dass sich Längen sinnvoll vergleichen lassen.

Ein Kreis behauptet, dass die Teile zusammen ein Ganzes ergeben.

Wer zuerst den Typ wählt, sucht danach Daten, die dazu passen.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Kapitel 1

Welches Diagramm?

Sechs Absichten und die Diagramme, die dazu passen

Sechs Absichten, sechs Familien



Verteilung

Wo sammelt sich die Masse?



Zusammenhang

Hängen zwei Grössen zusammen?



Vergleich

Wer ist grösser, wer kleiner?



Zusammensetzung

Woraus besteht ein Ganzes?



Entwicklung

Was verändert sich über die Zeit?



Verortung

Wo im Raum passiert es?

Zwei Fragen an dieselben Daten

ENTWICKLUNG

«Nimmt die Zahl der Hitzetage zu?»

Die Jahre folgen aufeinander, also verbindet die Linie sie zu Recht.

```
type: 'line'
```

VERGLEICH

«Welches waren die heissesten Sommer?»

Eine Rangliste hat keine Reihenfolge in der Zeit, also Balken.

```
type: 'bar'
```

Dieselben 258 Datensätze, zwei Absichten, zwei Diagrammtypen.

Diese Typen bringt Chart.js mit

Chart.js

Absicht	Diagramm	Chart.js
Vergleich	Bar Chart, liegend oder stehend	<code>type: 'bar'</code>
Entwicklung	Line Chart, mit oder ohne Fläche	<code>type: 'line'</code>
Zusammenhang	Scatterplot	<code>type: 'scatter'</code>
Zusammenhang	Bubble Chart	<code>type: 'bubble'</code>
Zusammensetzung	Pie Chart, Doughnut	<code>type: 'pie'</code>
Zusammensetzung	Stacked Bar Chart	<code>stacked: true</code>

Dazu kommen `radar` und `polarArea`, die man selten braucht.

Und diese Typen nicht

Chart.js

Diagramm	Warum nicht	Womit stattdessen
Histogram	Chart.js bildet keine Klassen	Klassen selbst zählen, dann <code>bar</code>
Boxplot	kein eingebauter Typ	Datawrapper oder Plugin
Dot Plot	kein eingebauter Typ	Datawrapper
Gantt Chart	nur über einen Umweg	Balken mit Start- und Endwert
Symbol Map	keine Karten	Leaflet
Choropleth Map	keine Karten	Leaflet

Braucht eure Story eine Karte, nehmt ihr Leaflet – dazu kommt Kapitel 5.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Kapitel 2

Von den Daten zum Diagramm

Der Bauplan und die eine wichtige Umformung

Ein Diagramm braucht drei Dinge

HTML + JS

IM HTML: DIE BIBLIOTHEK UND EINE ZEICHENFLÄCHE

```
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.5.1/dist/chart.umd.min.js"></script>
<canvas id="verlauf"></canvas>
```

IM JAVASCRIPT: DER AUFTRAG AN CHART.JS

```
const chart = new Chart(document.querySelector('#verlauf'), {
  type: 'line',           // welcher Diagrammtyp
  data: { labels: [], datasets: [] }, // was gezeigt wird
  options: verlaufOptions, // wie es aussieht
});
```

Chart.js braucht ein bestimmtes Datenformat

JavaScript

WAS DER ENDPUNKT LIEFERT

```
[  
  {"city": "Bern", "year": 1940, "hot_days": 0},  
  {"city": "Bern", "year": 1941, "hot_days": 1},  
  {"city": "Bern", "year": 1942, "hot_days": 0}  
]
```



WAS CHART.JS WILL

```
labels: [1940, 1941, 1942]  
data:   [0, 1, 0]
```

Zwei Listen, gleich lang und in derselben Reihenfolge.

Eine Stadt wird eine Linie

JavaScript

```
function yearsOf(rows) {  
  return [...new Set(rows.map((row) => row.year))].sort((a, b) => a - b);  
}  
  
function datasetFor(rows, city) {  
  const cityRows = rows.filter((row) => row.city === city);  
  
  return {  
    label: city,  
    data: cityRows.map((row) => row.hot_days),  
    borderColor: CITY_COLORS[city],  
  };  
}
```

Ein `dataset` ist eine Linie: die Zahlen und ihr Aussehen.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Kapitel 3

Die Daten holen

Der eigene Endpunkt, gelesen mit fetch()

Der Browser fragt euren eigenen Endpunkt ab

JavaScript

```
const response = await fetch('unload.php?city=Bern');  
const summers = await response.json();
```

Dieselben zwei Zeilen wie bei jeder fremden API im letzten Semester.

Neu ist nur, dass die API diesmal von euch stammt.

Der `$_GET`-Filter aus dem Endpunkt wird hier zum ersten Mal wirklich benutzt.

Nicht jede Antwort ist die erwartete

JavaScript

```
if (!response.ok) {  
  throw new Error(`Der Endpunkt antwortet mit Status ${response.status}.`);  
}  
  
const contentType = response.headers.get('content-type') ?? '';  
  
if (!contentType.includes('application/json')) {  
  throw new Error('Die Antwort ist kein JSON.');
```

`fetch()` wirft bei Status 404 oder 500 keinen Fehler – es ist ja eine Antwort angekommen.

Öffnet ihr die Seite nicht über eure Domain, kommt statt JSON etwas anderes zurück.

Die Übergabe ist eine einzige Zeile

JavaScript

BEIM BAUEN

```
const DATEN_URL = MUSTERDATEN;    // data/heat-summers.json
```

NACH DER ÜBERGABE

```
const DATEN_URL = ENDPUNKT;       // unload.php
```

Das Frontend-Team baut mit einer Musterdatei, lange bevor das Backend fertig ist.

Am Schluss wechselt eine Zeile, und dem Diagramm sieht man nichts an.

Das funktioniert nur, weil beide Seiten denselben Datenvertrag eingehalten haben.

Extract → Transform → Load → Datenbank → Unload → Chart.js

Kapitel 4

Auf Bedienung reagieren

Aktualisieren statt neu erzeugen

Einmal erzeugen, danach aktualisieren

Chart.js

```
const chart = new Chart(canvas, { type: 'line', data: leer, options });

function render() {
  chart.data.labels = yearsOf(summers);
  chart.data.datasets = cityNamesOf(summers).map((city) => datasetFor(summers, city));
  chart.update();
}
```

Ein zweites `new Chart()` auf demselben Canvas meldet «Canvas is already in use».

Zwei Sorten Interaktion

Der Server antwortet neu

Die Stadtauswahl schickt eine neue Anfrage.

```
change → fetch('unload.php?city=Bern')  
→ SELECT → render()
```

Der Browser rechnet selbst

Der Zeitraum-Schieber benutzt Daten, die schon da sind.

```
input → fromYear ändern  
→ render()
```

Der Server filtert, was viel ist – der Browser filtert, was schon da ist.

Ein Zustand, eine Zeichenfunktion

JavaScript

```
let summers = [];           // was geladen ist
let fromYear = 1940;        // welcher Ausschnitt
let metric = 'hot_days';    // welcher Messwert
```

Jedes Bedienelement ändert genau eine dieser Variablen und ruft dann `render()` auf.

So steuern drei Bedienelemente zwei Diagramme, ohne dass der Code unübersichtlich wird.

`render()` ist die einzige Stelle, die ein Diagramm anfassen darf.

Extract → Transform → Load → Datenbank → Unload → Leaflet

Kapitel 5

Karten mit Leaflet

Dieselbe Kette, eine andere Zeichenbibliothek

Leaflet braucht nur drei Dinge

HTML + JS

IM HTML: ZWEI DATEIEN UND EIN BEHÄLTER MIT HÖHE

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/leaflet@1.9.4/dist/leaflet.css">
<script src="https://cdn.jsdelivr.net/npm/leaflet@1.9.4/dist/leaflet.js"></script>
<div id="karte"></div>      /* im CSS: height: 520px */
```

IM JAVASCRIPT: AUSSCHNITT, HINTERGRUND, DATEN

```
const karte = L.map('karte').setView([20, 0], 2);
L.tileLayer(kachelUrl, { attribution: quelle }).addTo(karte);
L.geoJSON(grenzen, { style: stilFuer }).addTo(karte);
```

Ohne Höhe im CSS bleibt die Karte unsichtbar, und die Konsole meldet nichts.

Die Umrisse der Länder sind GeoJSON

Leaflet kennt keine Länder, es liefert nur den Hintergrund.

LAENDER.GEOJSON, EIN EINTRAG PRO LAND

```
{
  "type": "Feature",
  "properties": { "iso3": "AUS", "name": "Australia" },
  "geometry": {
    "type": "Polygon",
    "coordinates": [[[145.4, -40.8], [146.8, -41.0], ...]]
  }
}
```

`geometry` ist der Umriss als Liste von Punkten, `properties` sind die Angaben zum Land.

WOHER IHR DIE DATEI BEKOMMT

- Weltkarte: Natural Earth, gratis und frei nutzbar
- Schweiz: swisstopo, Kantone und Gemeinden
- Sonst: nach «<Gebiet> geojson» suchen

Auch eine Kartendatei ist ein Datensatz, den ihr kürzen dürft.

Verbunden wird über einen Schlüssel

JavaScript

WAS EUER ENDPUNKT LIEFERT

```
{
  "country": "USA",
  "iso3": "USA",
  "incidents": 1486,
  "top_species": "White shark"
}
```



WAS IN DER KARTENDATEI STEHT

```
{
  "properties": {
    "iso3": "USA",
    "name": "United States of America"
  }
}
```

Zwei Dateien aus verschiedenen Quellen passen zusammen, weil beide denselben Ländercode kennen.

Den Ländercode legt eine Tabelle fest

Im Rohdatensatz steht nur ein Name, und Namen sind Schreibweisen.

LAENDER_ISO.JSON: NAME → CODE

```
{  
  "USA": "USA",  
  "AUSTRALIA": "AUS",  
  "SOUTH AFRICA": "ZAF",  
  "PAPUA NEW GUINEA": "PNG"  
}
```

Die Tabelle entsteht aus den häufigsten Rohwerten, nicht aus dem Gedächtnis.

NACHGESCHLAGEN WIRD BEIM TRANSFORM

```
$value = strtoupper(trim($raw));  
  
return $isoByCountry[$value] ?? null;
```

Fehlt ein Name in der Tabelle, bleibt `null` stehen.

Ohne Ländercode findet das Land seine Fläche nicht und bleibt grau.

Die Klassengrenzen sind eine Entscheidung

Leaflet

DIE DATEN

USA	1486
AUSTRALIA	540
SOUTH AFRICA	322
...	
über 60 Länder	1

Ein linearer Farbverlauf färbt nur die USA dunkel.

DARUM KLASSEN

```
const KLASSEN = [  
  { ab: 500, farbe: '#2f6b7a' },  
  { ab: 100, farbe: '#4b93a4' },  
  { ab: 50, farbe: '#7db0bd' },  
  { ab: 20, farbe: '#a8c9d2' },  
  { ab: 5, farbe: '#cfe0e5' },  
  { ab: 1, farbe: '#eef4f6' },  
];
```

Wer die Grenzen verschiebt, ändert die Aussage der Karte, ohne eine Zahl anzufassen.

Eine Weltkarte sieht aus wie die Wahrheit

- Grau heisst «keine Zahl», nicht «nichts passiert»
- Grosse Länder fallen auf, auch wenn dort wenig passiert
- Kleine Inselstaaten verschwinden, obwohl sie zuoberst stehen
- Wo niemand zählt, bleibt die Karte hell

Was die Karte nicht zeigt, gehört als Satz unter die Karte.

Übertragt das Gelernte auf eure Grafik

- Die Absicht der Grafik in einem Satz benennen
- Zwei Diagrammtypen vergleichen, bevor ihr einen wählt
- Prüfen, ob Chart.js diesen Typ überhaupt kann
- Achsen beschriften und die Einheit dazuschreiben
- Das Diagramm einmal erzeugen und danach aktualisieren
- Ladefehler und leere Daten sichtbar melden
- Quelle, Zeitraum und Grenze der Aussage danebenstellen
- Bei einer Karte dazuschreiben, was sie nicht zeigt

Ein Diagramm, das seine Aussage ohne Klick zeigt, ist besser als eines, das sie versteckt.

Was bleibt

Die Absicht wählt den Diagrammtyp.

`labels` und `data` füllen ihn.

`render()` hält das Bild aktuell.

Die Bibliothek ist austauschbar, die Kette davor nicht.

Nächster Schritt: im Code-Along selbst bauen